

N-Reinas con Backtracking

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Problema	2
1.1. Número de soluciones	2
2. Algoritmo de Backtracking	2
2.1. Estrategia	2
2.2. Implementación	2
2.3. Verificación de seguridad	2
3. Visualización en Tiempo Real	3
4. Representación del Tablero	3
5. Complejidad	3
6. Interfaz Gráfica	3

1. Descripción del Problema

El problema de las **N-Reinas** consiste en colocar N reinas en un tablero de $N \times N$ de forma que ninguna reina ataque a otra. Dos reinas se atacan si comparten fila, columna o diagonal.

1.1. Número de soluciones

N	Soluciones
4	2
5	10
6	4
7	40
8	92
11	2,680
14	365,596

2. Algoritmo de Backtracking

2.1. Estrategia

Se coloca una reina por fila. Para cada fila se prueba cada columna; si la posición es segura, se avanza a la siguiente fila. Si ninguna columna es válida, se retrocede (backtrack) a la fila anterior.

2.2. Implementación

```
1 void colocarReina(int fila) {
2     if (fila == N) {
3         soluciones++;
4         guardarSolucion(tablero.clone());
5         return;
6     }
7     for (int col = 0; col < N; col++) {
8         if (esSeguro(fila, col)) {
9             tablero[fila] = col; // reina en (fila,col)
10            colocarReina(fila + 1);
11            tablero[fila] = -1; // backtrack
12        }
13    }
14 }
```

2.3. Verificación de seguridad

Una posición ($fila, col$) es segura si ninguna reina ya colocada la ataca:

```
1 boolean esSeguro(int fila, int col) {
2     for (int f = 0; f < fila; f++) {
3         int c = tablero[f];
4         // misma columna
```

```

5         if (c == col) return false;
6         // diagonal (diferencia de fila == diferencia de col)
7         if (Math.abs(f - fila) == Math.abs(c - col))
8             return false;
9     }
10    return true;
11 }

```

No es necesario verificar filas porque se coloca exactamente una reina por fila.

3. Visualización en Tiempo Real

El visualizador muestra cada paso del backtracking:

- **Colocación:** se dibuja la reina en color verde.
- **Backtrack:** la reina se elimina y se muestra en rojo momentáneamente.
- **Solución encontrada:** todo el tablero se resalta en amarillo.

La animación usa un `javax.swing.Timer` con delay configurable (10ms a 500ms) para controlar la velocidad de visualización.

4. Representación del Tablero

El tablero se representa como un arreglo `int[N]` donde `tablero[fila] = columna` indica que la reina de esa fila está en esa columna. Esto garantiza exactamente una reina por fila desde el diseño de la estructura de datos.

5. Complejidad

- **Peor caso sin poda:** $O(N^N)$
- **Con backtracking:** se poda enormemente el árbol. Para $N = 8$ solo se exploran $\sim 15,720$ nodos de los $8^8 = 16,777,216$ posibles.
- **Optimización:** se pueden usar conjuntos de bits para verificar columnas y diagonales en $O(1)$.

6. Interfaz Gráfica

- El tablero se dibuja con `paintComponent()`, alternando colores claro/oscuro.
- Las reinas se representan con el símbolo Q o con iconos personalizados.
- Selector de tamaño N (de 4 a 14).
- Contador de soluciones encontradas en tiempo real.